

ESP32 Pre-Impact Fall Detection

AI-Free Baseline Report, Hardware Test and Future AI Integration Plan

This report documents the completed AI-free physical baseline for an ESP32-based fall detection system and explains how the trained AI model will be integrated later.

1. Project Objective

The objective is to use a waist-mounted MPU6050 inertial sensor with an ESP32 to identify a possible fall and activate an LED and buzzer alert. Before AI integration, a deterministic rule-based baseline is used to verify the physical sensor, wiring, sampling and alarm path.

Free-fall / unloading → impact or rapid rotation → possible-fall alert

This baseline is not a trained machine-learning classifier. It is an engineering validation stage for the physical project.

2. Completed Baseline Functions

Function	Status
ESP32 firmware upload	Completed
MPU6050 I2C communication	Completed
I2C address detection at 0x68	Completed
Acceleration and gyroscope reading	Completed
50 Hz sampling	Completed
Serial Monitor output	Completed
LED output	Completed
Active buzzer output	Completed
Manual serial commands	Completed

3. Hardware Configuration

Component	Purpose
ESP32 DevKit V1	Main microcontroller
MPU6050 / GY-521	Three-axis accelerometer and three-axis gyroscope
Active buzzer	Audible alarm
Red LED	Visual alarm/status indicator
220 ohm resistor	LED current limiting
USB data cable	Power and programming

4. Direct Pin Connections

MPU6050 VCC → ESP32 3V3
MPU6050 GND → ESP32 GND
MPU6050 SDA → ESP32 GPIO21 / D21
MPU6050 SCL → ESP32 GPIO22 / D22
MPU6050 AD0 → GND (I2C address 0x68)

Buzzer + → ESP32 GPIO4 / D4
Buzzer - → ESP32 GND

ESP32 GPIO2 / D2 → 220 ohm resistor → LED anode
LED cathode → ESP32 GND

All grounds must be common. The first hardware test should use USB power. A LiPo and TP4056 should be added only after the USB-powered test is successful.

5. Rule-Based Detection Logic

Parameter	Value
-----------	-------

Sampling interval	20 ms
Sampling frequency	50 Hz
Free-fall threshold	0.65 G
Impact threshold	2.50 G
Rotation threshold	250 degree/s
Minimum free-fall duration	80 ms
Confirmation window	1.5 seconds
Alarm duration	3 seconds
Alarm cooldown	5 seconds

The detector first looks for a low-acceleration/free-fall condition. It then waits for an impact or rapid rotation inside the confirmation window. If both stages are satisfied, the alarm is activated.

6. Test Cases

Test case	Action	Expected result
Stationary test	Keep the sensor still	Acceleration magnitude near 1 G; no alarm
Normal movement	Move the sensor slowly	Sensor values change; normally no alarm
Rotation test	Rotate the sensor	Gyroscope values change
Manual LED/buzzer test	Send test in Serial Monitor	LED and buzzer activate three times
Manual alarm test	Send alarm	Alarm turns on
Manual stop test	Send off	Alarm turns off
Status test	Send status	Current detector state is printed
Safe movement test	Perform controlled movement over a soft surface	Possible alert only after the rule sequence is satisfied

7. Serial Monitor Commands

```
test  = blink LED and beep buzzer three times
alarm = force the alarm on
off   = turn the alarm off
status = show sensor and alarm status
reset = reset detector state
help  = show the command list
```

Serial Monitor settings:

Port: the ESP32 COM port, for example COM3
Baud rate: 115200

8. Arduino IDE Setup

- Install Arduino IDE.
- Install ESP32 board support using the Espressif Boards Manager URL.
- Select Board → esp32 → ESP32 Dev Module.
- Select the ESP32 COM port.
- Open esp32_rule_based_fall_detector.ino.
- Install the required libraries if using the Adafruit version; the supplied AI-free direct-register code only requires the ESP32 core and Wire.
- Click Verify, then Upload.
- Open Tools → Serial Monitor and select 115200 baud.

9. Full AI-Free Firmware

The following is the final AI-free rule-based firmware used for the initial physical test.

```
#include <Arduino.h>
#include <Wire.h>
#include <math.h>

// AI-free rule-based fall detector
// Human sensor input + Serial Monitor manual test commands

const uint8_t MPU6050_ADDR = 0x68;

const int SDA_PIN = 21;
const int SCL_PIN = 22;
const int BUZZER_PIN = 4;
const int LED_PIN = 2;

const float FREEFALL_THRESHOLD_G = 0.65f;
const float IMPACT_THRESHOLD_G = 2.50f;
const float ROTATION_THRESHOLD_DPS = 250.0f;

const unsigned long SAMPLE_INTERVAL_MS = 20;
const unsigned long FREEFALL_MIN_MS = 80;
const unsigned long FALL_CONFIRM_WINDOW_MS = 1500;
const unsigned long ALARM_DURATION_MS = 3000;
const unsigned long ALARM_COOLDOWN_MS = 5000;

unsigned long last_sample_time = 0;
unsigned long freefall_start_time = 0;
unsigned long alarm_start_time = 0;
unsigned long last_alarm_time = 0;

bool sensor_ready = false;
bool freefall_detected = false;
bool alarm_active = false;

void write_register(uint8_t reg, uint8_t value) {
    Wire.beginTransmission(MPU6050_ADDR);
```

```

Wire.write(reg);
Wire.write(value);
Wire.endTransmission(true);
}

uint8_t read_register(uint8_t reg) {
Wire.beginTransmission(MPU6050_ADDR);
Wire.write(reg);
if (Wire.endTransmission(false) != 0) return 0xFF;
if (Wire.requestFrom(MPU6050_ADDR, (uint8_t)1, (uint8_t)true) != 1) return 0xFF;
return Wire.read();
}

bool i2c_exists(uint8_t address) {
Wire.beginTransmission(address);
return Wire.endTransmission() == 0;
}

void scan_i2c_bus() {
Serial.println("Scanning I2C bus...");
int found = 0;

for (uint8_t address = 1; address < 127; address++) {
if (i2c_exists(address)) {
Serial.print("I2C device found at address: 0x");
if (address < 16) Serial.print("0");
Serial.println(address, HEX);
found++;
}
}

Serial.print("Total I2C devices found: ");
Serial.println(found);
}

bool initialize_mpu6050() {
if (!i2c_exists(MPU6050_ADDR)) {
return false;
}

Serial.print("WHO_AM_I: 0x");
uint8_t who = read_register(0x75);
if (who < 16) Serial.print("0");
Serial.println(who, HEX);

// Wake sensor
write_register(0x6B, 0x00);

// Accelerometer +/-16G
write_register(0x1C, 0x18);

// Gyroscope +/-2000 DPS
write_register(0x1B, 0x18);

// Digital low-pass filter
write_register(0x1A, 0x04);

// 50 Hz sample rate
write_register(0x19, 19);

delay(100);
return true;
}

int16_t read_int16() {
uint8_t high_byte = Wire.read();
uint8_t low_byte = Wire.read();
return (int16_t)((high_byte << 8) | low_byte);
}

bool read_sensor(
float &ax,
float &ay,
float &az,
float &gx,
float &gy,
float &gz
) {
Wire.beginTransmission(MPU6050_ADDR);
Wire.write(0x3B);

if (Wire.endTransmission(false) != 0) return false;
if (Wire.requestFrom(MPU6050_ADDR, (uint8_t)14, (uint8_t)true) != 14) return false;

int16_t raw_ax = read_int16();
int16_t raw_ay = read_int16();
int16_t raw_az = read_int16();
int16_t raw_temp = read_int16();
int16_t raw_gx = read_int16();
int16_t raw_gy = read_int16();
int16_t raw_gz = read_int16();

(void)raw_temp;

// +/-16G = 2048 LSB/G
ax = (float)raw_ax / 2048.0f;
ay = (float)raw_ay / 2048.0f;
az = (float)raw_az / 2048.0f;

// +/-2000 DPS = 16.4 LSB/DPS
gx = (float)raw_gx / 16.4f;
gy = (float)raw_gy / 16.4f;
gz = (float)raw_gz / 16.4f;

return true;
}

void start_alarm(const char *reason) {
if (alarm_active) return;

alarm_active = true;
}

```

```

alarm_start_time = millis();
last_alarm_time = alarm_start_time;

digitalWrite(LED_PIN, HIGH);
digitalWrite(BUZZER_PIN, HIGH);

Serial.println();
Serial.println("=====");
Serial.print("FALL ALERT: ");
Serial.println(reason);
Serial.println("LED = ON");
Serial.println("BUZZER = ON");
Serial.println("=====");
}

void stop_alarm() {
  alarm_active = false;
  digitalWrite(LED_PIN, LOW);
  digitalWrite(BUZZER_PIN, LOW);
  Serial.println("ALARM: OFF");
}

void reset_detector() {
  freefall_detected = false;
  freefall_start_time = 0;
  stop_alarm();
  Serial.println("Detector state reset.");
}

void manual_alarm_test() {
  Serial.println("Manual LED/buzzer test started.");

  for (int i = 0; i < 3; i++) {
    digitalWrite(LED_PIN, HIGH);
    digitalWrite(BUZZER_PIN, HIGH);
    delay(300);
    digitalWrite(LED_PIN, LOW);
    digitalWrite(BUZZER_PIN, LOW);
    delay(300);
  }

  Serial.println("Manual LED/buzzer test finished.");
}

void print_help() {
  Serial.println();
  Serial.println("Serial commands:");
  Serial.println(" test = blink LED and beep buzzer 3 times");
  Serial.println(" alarm = force fall alert ON");
  Serial.println(" off = turn alarm OFF");
  Serial.println(" fall = simulate a possible fall event");
  Serial.println(" reset = reset detector state");
  Serial.println(" status = show detector status");
  Serial.println(" help = show commands");
  Serial.println();
}

void handle_serial_commands() {
  if (!Serial.available()) return;

  String command = Serial.readStringUntil('\n');
  command.trim();
  command.toLowerCase();

  if (command == "test" || command == "t") {
    manual_alarm_test();
  }
  else if (command == "alarm" || command == "a" || command == "fall" || command == "f") {
    start_alarm("SERIAL MANUAL TEST");
  }
  else if (command == "off" || command == "o") {
    stop_alarm();
  }
  else if (command == "reset" || command == "r") {
    reset_detector();
  }
  else if (command == "status" || command == "s") {
    Serial.print("Sensor ready: ");
    Serial.println(sensor_ready ? "YES" : "NO");
    Serial.print("Free-fall state: ");
    Serial.println(freefall_detected ? "ACTIVE" : "IDLE");
    Serial.print("Alarm state: ");
    Serial.println(alarm_active ? "ON" : "OFF");
  }
  else if (command == "help" || command == "h") {
    print_help();
  }
  else if (command.length() > 0) {
    Serial.print("Unknown command: ");
    Serial.println(command);
    print_help();
  }
}

void setup() {
  Serial.begin(115200);
  Serial.setTimeout(100);
  delay(1500);

  pinMode(BUZZER_PIN, OUTPUT);
  pinMode(LED_PIN, OUTPUT);
  digitalWrite(BUZZER_PIN, LOW);
  digitalWrite(LED_PIN, LOW);

  Serial.println();
  Serial.println("=====");
  Serial.println("RULE-BASED FALL DETECTOR");
  Serial.println("AI MODEL: DISABLED");
  Serial.println("=====");

  Wire.begin(SDA_PIN, SCL_PIN);

```

```

Wire.setClock(400000);

Serial.println("SDA: GPIO 21");
Serial.println("SCL: GPIO 22");
Serial.println("MPU6050 address: 0x68");

scan_i2c_bus();

if (!initialize_mpu6050()) {
    Serial.println("ERROR: MPU6050 initialization failed");

    while (true) {
        digitalWrite(LED_PIN, HIGH);
        delay(250);
        digitalWrite(LED_PIN, LOW);
        delay(250);
    }
}

sensor_ready = true;

Serial.println("MPU6050 READY");
Serial.println("Rule: free-fall followed by impact/rotation");
Serial.println("System ready.");

print_help();
}

void loop() {
    handle_serial_commands();

    if (!sensor_ready) {
        delay(1000);
        return;
    }

    unsigned long now = millis();

    if (now - last_sample_time < SAMPLE_INTERVAL_MS) {
        return;
    }

    last_sample_time = now;

    float ax, ay, az, gx, gy, gz;

    if (!read_sensor(ax, ay, az, gx, gy, gz)) {
        Serial.println("ERROR: MPU6050 read failed");
        return;
    }

    float acceleration_magnitude = sqrt(
        ax * ax +
        ay * ay +
        az * az
    );

    float rotation_magnitude = sqrt(
        gx * gx +
        gy * gy +
        gz * gz
    );

    Serial.print("ax=");
    Serial.print(ax, 3);
    Serial.print(" ay=");
    Serial.print(ay, 3);
    Serial.print(" az=");
    Serial.print(az, 3);
    Serial.print(" gx=");
    Serial.print(gx, 1);
    Serial.print(" gy=");
    Serial.print(gy, 1);
    Serial.print(" gz=");
    Serial.print(gz, 1);
    Serial.print(" | A=");
    Serial.print(acceleration_magnitude, 3);
    Serial.print(" G | R=");
    Serial.print(rotation_magnitude, 1);
    Serial.println(" DPS");

    // Stage 1: possible free-fall/unloading
    if (acceleration_magnitude <= FREEFALL_THRESHOLD_G) {
        if (!freefall_detected) {
            freefall_detected = true;
            freefall_start_time = now;
            Serial.println("Possible free-fall detected; waiting for confirmation...");
        }
    }

    // Stage 2: confirm with impact or rapid rotation
    if (freefall_detected) {
        unsigned long freefall_duration =
            now - freefall_start_time;

        bool freefall_long_enough =
            freefall_duration >= FREEFALL_MIN_MS;

        bool within_confirmation_window =
            freefall_duration <= FALL_CONFIRM_WINDOW_MS;

        bool impact_detected =
            acceleration_magnitude >= IMPACT_THRESHOLD_G;

        bool rotation_detected =
            rotation_magnitude >= ROTATION_THRESHOLD_DPS;

        if (
            freefall_long_enough &&
            within_confirmation_window &&
            (impact_detected || rotation_detected) &&

```

```

    !alarm_active &&
    now - last_alarm_time >= ALARM_COOLDOWN_MS
  ) {
    start_alarm("FREE-FALL + IMPACT/ROTATION CONFIRMED");
    freefall_detected = false;
  }

  if (!within_confirmation_window) {
    freefall_detected = false;
    Serial.println("Fall confirmation window expired.");
  }
}

if (
  alarm_active &&
  now - alarm_start_time >= ALARM_DURATION_MS
) {
  stop_alarm();
}
}

```

10. How AI Will Be Added Later

The current hardware wiring does not need to change. The live MPU6050 reader will remain, but instead of sending the signal only to the rule-based detector, the firmware will place the six channels into the trained model input window.

Current baseline:
MPU6050 → rule logic → LED/buzzer

AI version:
MPU6050 → 100 x 6 ring buffer → frozen normalization
→ INT8 quantization → TensorFlow Lite Micro model
→ non-fall / alert / fall probability → LED/buzzer

AI integration files:

model.tflite
model_data.h
model_config.h
TensorFlowLite_ESP32 library
AI-enabled ESP32 firmware

AI integration sequence:

1. Keep the direct MPU6050 reader that has already passed the physical test.
2. Collect 100 samples at 50 Hz.
3. Apply the frozen channel mean and standard deviation from training.
4. Quantize the normalized values using the TFLite input scale and zero point.
5. Invoke the INT8 TensorFlow Lite Micro model.
6. Read the three output probabilities.
7. Trigger the alarm when alert or fall probability reaches the configured threshold.
8. Compare physical model output with the Wokwi replay output.

11. Current Limitations

- The AI-free detector is a heuristic baseline and does not represent the final trained model accuracy.
- A threshold detector can generate false alarms during vigorous movement.
- A simulated or controlled fall is not equivalent to an uncontrolled real-world fall.
- The physical device still requires measured latency, RAM, battery life and false-alarm evaluation.
- The AI model should only be enabled after the baseline hardware test remains stable.

12. Final Status

AI-free physical test: COMPLETE
MPU6050 live readings: COMPLETE
LED/buzzer manual test: COMPLETE
Rule-based live test: READY
Trained AI integration: NEXT STAGE

Conclusion: The AI-free code is the final baseline for confirming that the physical ESP32 and MPU6050 project works. The trained AI model can now be added without changing the basic sensor wiring.